

Portfolio — Tech Stack Documentation

Developer: Soorya Durairaj

Project: Personal Portfolio Website

Stack: React · Vite · Framer Motion · GSAP · Lenis · Lucide · CSS

Date: March 2026

1. Project Overview

This is a fully custom, single-page portfolio website built to showcase Soorya Durairaj's skills as a Flutter developer and cloud engineer. The portfolio presents:

- **Professional experience** at Tata Consultancy Services (BT Telecom) and a Flutter internship
- **4 complete Flutter mobile applications** built with REST APIs, GraphQL, Firebase, and GCP
- **Technical skill set** using 12 technology icons in an interactive Bento Grid layout
- **4 cloud and AI certifications**, including Associate Google Cloud Engineer
- **Contact form** wired to Gmail via `mailto:` for direct inquiry

The site is a premium, animation-heavy, fully responsive web experience inspired by the Portavia Framer template, rebuilt entirely in React and Vite.

2. Frontend Technologies

Technology	Version	Role
React	19	Component-based UI framework
Vite	7	Lightning-fast build tool and dev server
Framer Motion	12	Declarative animations, scroll-triggered effects, <code>AnimatePresence</code>
GSAP	3	Scroll-synced 3D flip card animation (Hero → Services transition)
@gsap/react	2	React hook integration for GSAP (<i>useGSAP</i>)
Lenis	1	Buttery-smooth momentum scroll (replaces native scroll)
Lucide React	latest	Consistent, lightweight SVG icon set
Vanilla CSS	—	Full custom styling with CSS variables, no framework

Why Vite over CRA? Vite uses native ES modules in development for near-instant HMR (Hot Module Replacement), making the developer experience significantly faster.

Why Framer Motion + GSAP together? Framer Motion handles React-aware animations (component entrance/exit, `whileInView`, `AnimatePresence`). GSAP handles the complex scroll-scrubbed 3D card flip between the Hero and Services sections, which requires precise rect-based positional interpolation not natively available in Framer Motion.

3. UI Design Approach

Bento Grid Layout

The Technical Skills section uses CSS `grid-template-areas` to define a named Bento Grid — a design pattern inspired by Apple's product pages and modern developer portfolios. Featured technologies (Flutter, Firebase, Google Cloud) span 2 columns (large cards) while other skills use single-column cards.

Glassmorphism

All skill cards use the glassmorphism design pattern:

- `backdrop-filter: blur(18px) saturate(160%)` — frosted glass appearance
- Semi-transparent `rgba(255, 255, 255, 0.55)` `background` (light mode)
- Bright inset border: `inset 0 1px 0 rgba(255, 255, 255, 0.8)`
- Per-card ambient glow orb using brand-specific accent colors

Dark Mode / Light Mode

A `isDark` state in `App.jsx` toggles `body.dark` class. All CSS variables are defined in `:root` (light) and `body.dark` (dark) blocks in `index.css`. The `ThemeToggle` button allows users to switch modes at any time with a smooth `0.4s` color transition.

Responsive Design

All sections use `clamp()` for fluid typography and CSS Grid + media queries at 900px and 600px breakpoints for layout reflow. The Bento Grid collapses from 4-column to 2-column on tablet and mobile.

Typography

Uses Google Fonts **Inter** (weights 300–900) — the industry standard for clean, modern UI typography with excellent legibility at both display and body sizes.

4. Animation & Interaction

GSAP 3D Flip Card (Hero → Services)

The most complex animation in the portfolio. As the user scrolls from the Hero section through the Services section:

1. A floating `div` with `perspective-3d` and `rotateY: 180` is animated using `gsap.timeline + ScrollTrigger`
2. The card tracks from the hero portrait position to the services image position using live `getBoundingClientRect()` calculations
3. This creates a cinematic 3D page-flip effect entirely driven by scroll position

Framer Motion `whileInView`

Every section uses `whileInView` with `once: true` and `staggered` `delay` values so content gracefully fades and slides in as the user scrolls, without re-triggering on scroll-back.

Floating Icon Animation (Technical Skills)

Each skill card icon floats continuously with `@keyframes bentoFloat` — a gentle `0 → -7px → 0` vertical oscillation. Each icon has a unique `animationDuration` (3.2s–4.0s) and `animationDelay` (based on `array index × 0.28s`), so the icons float naturally out of phase with each other.

Cursor Spotlight Effect

Each skill card tracks `mousemove` events in React. The cursor `x/y` position within the card is written directly to CSS custom properties (`--x`, `--y`) on the DOM element via `element.style.setProperty()`. A `.bento-spotlight` pseudo-layer renders a `radial-gradient(circle at var(--x) var(--y))` that follows the cursor. The `--spotlight-opacity` variable fades to `0` on `mouseleave`.

Brand-Color Icon Glow

Each technology has a `--icon-glow` CSS variable (RGB values only, e.g. `54,190,247` for Flutter). On card hover, the `.bento-icon` receives:

```
filter: drop-shadow(0 0 8px rgba(var(--icon-glow), 0.65))
       drop-shadow(0 0 16px rgba(var(--icon-glow), 0.3));
```

This produces a tight inner glow plus a wider outer halo specific to each brand color.

Stacked Project Cards

The Projects section uses `Framer Motion's useScroll + useTransform` to create a stacked scroll effect — each card scales down slightly as the next card scrolls into place on top of it, creating a layered deck effect.

FAQ & Services Accordions

Both use `AnimatePresence + motion.div` with `height: 0 → auto` to animate the open/close of accordion bodies. The `ChevronUp` icon rotates `180°` on open using `motion.div animate={{ rotate }}`.

5. Technical Skills Section

Implementation

The Technical Skills section (`TechnicalSkills.jsx + TechnicalSkills.css`) renders 12 technology cards in a Bento Grid layout. Each card:

- 1. Uses `motion.div` for scroll-triggered reveal animation
- 2. Tracks mouse position for the cursor spotlight via `useRef + onMouseMove`
- 3. Displays the official technology icon from the jsDelivr CDN (GitHub: devicons/devicon)
- 4. Shows a hover tooltip with skill name, category, and experience

Skills Displayed & Why

Technology	Category	Reason
Flutter	Mobile Framework	Primary specialization — 4 apps built
Dart	Core Language	Flutter's native language
Firebase	Backend / Auth	Used in Pocket Giving charity app
GraphQL	API Layer	Used with Hasura in movie streaming & food ordering apps
Git	Version Control	Industry-standard SCM used in all projects
GitHub	Code Hosting	All repositories and CI/CD pipelines
Postman	API Testing	REST API development and debugging
Figma	UI/UX Design	Designing screens from wireframes to implementation
VS Code	IDE	Primary development environment
Google Cloud	Cloud Platform	Associate Cloud Engineer certified
Cloudinary	Media Management	Image/video CDN for mobile apps
JSON	Data Format	Core data exchange format in all API integrations

6. Performance Optimization

GPU-Friendly Animations

All animations use `transform` and `opacity` exclusively — properties that the browser composites on the GPU without triggering layout reflow. `will-change: opacity` is applied sparingly only on the spotlight overlay.

Lightweight Assets

Technology icons are loaded from the jsDelivr CDN as SVGs — vector format with no quality loss, typically under 5 KB each. Hero and service images are local `.jpg` files optimized for web.

loading="lazy" on Icons

All 12 skill icon `<img>` elements use `loading="lazy"` to defer off-screen image fetches until the user scrolls to the section.

Efficient Component Architecture

- `useCallback` wraps all event handlers (`mousemove/mouseleave`) on skill cards to prevent function recreation on re-render
- The `useCountUp` hook in About section uses `requestAnimationFrame` directly for smooth, JS-thread-friendly number counting
- GSAP's `ScrollTrigger` is synchronized with `Lenis` via `lenis.on('scroll', ScrollTrigger.update)` to avoid double-rendering scroll callbacks

Lenis Smooth Scroll

Lenis overrides the native scroll with a physics-based lerp interpolation (`lerp: 0.08`) to create buttery smooth momentum scroll, which also significantly improves the perceived performance of scroll-driven animations.

7. Deployment

Recommended Hosting

The portfolio is designed for deployment on **Vercel** or **Netlify** with zero-configuration:

```
# Build command
npm run build

# Output directory
dist/
```

Both platforms support:

- Automatic HTTPS via Let's Encrypt
- Global CDN edge network
- Continuous deployment from GitHub (push to main → auto deploy)
- Custom domain mapping

GitHub Repository

Source code is version-controlled on GitHub. The `.gitignore` excludes `node_modules/`, `dist/`, and environment files. The `README.md` provides setup instructions.

Development

```
npm install    # Install dependencies
npm run dev    # Start dev server at http://localhost:5173
npm run build  # Production build
npm run preview # Preview production build locally
```

8. Developer Explanation (Interview Style)

"This portfolio is a React + Vite single-page application that I built from scratch to showcase my Flutter development skills in a visually compelling way. I deliberately avoided UI frameworks like Tailwind or Material UI — all styling is vanilla CSS with CSS custom properties, which gives me fine-grained control over the dark/light theme system and allows every component to be precisely styled to match the design system.

The most technically interesting part is the scroll-driven 3D flip card between the Hero and Services sections, which I built using GSAP ScrollTrigger and live `getBoundingClientRect()` calculations. The card physically tracks from one DOM element to another as the user scrolls — it's essentially a scroll-scrubbed FLIP animation.

For the Technical Skills section, I designed a Bento Grid layout using named CSS grid-template-areas, with glassmorphism cards that include a cursor spotlight effect. The spotlight works by listening to mousemove events in React, writing cursor position into CSS custom properties on the DOM node, and using those variables in a radial-gradient that updates without any React re-renders — it's a pure DOM style mutation for maximum performance.

I also use Framer Motion extensively for all scroll-triggered entrance animations and the accordion components. Lenis provides momentum scroll that integrates with both GSAP ScrollTrigger and Framer Motion's scroll tracking. Every animation uses only transform and opacity to stay GPU-composited.

The result is a portfolio that demonstrates not just my Flutter skills but also my ability to build premium, production-quality web interfaces with advanced animation and interaction design."